UNIVERSITY OF
PLYMOUTH

In collaboration with Peninsula College

School of Technology & Engineering

BSc (Hons) Computer Science (Cyber Security)

(N/0613/6/0002)(04/2027)(MQA/PA15604)

MAL2018 – Information Management & Retrieval

Author:

BSCS2509254

Module Leader:

Dr. Ang Jin Sheng

Nov 19, 2025

Table of Contents

WordCount : <u>2724 words</u>

Code Repository

Github	https://github.com/OoiWeiChyeh/MAL2018_Assesment Module.git

1.0 Introduction

Coursework 1 (CW1) submission is focused on the analysis and relational database design for the TrailService micro-service within the context of the Trail Application. By using sample of trail data such as the Plymbridge Circular route, the exercise show how raw information can be modelled, normalised and stored to support the data management. The work starts by identifying the core data items required for the micro-service and applying the full normalization process from Unnormalised form(UNF) to Third Normal Form(3NF). A partial Entity Relationship Diagram(ERD) is provided to represent the results of the normalization process.

Nontheless, the coursework progresses into the design and implementation stages. A final ERD is created to show the full data model for the micro-service which mix the partial ERD with additional structures justified by proper assumptions. The final tasks got implementing part of this design In Microsoft SQL Server which have table creation, a link entity, a view, stored procedures and a trigger. When the SQL deployment and sample data would confirm the database design successfully supports the needs of the TrailService.

2.0 Background

The Trail application is intended to be a wellbeing focused platform that helps people find outdoor trails and hopefully spend more time in nature to possibly build habit where support physical and mental health. Instead of depending on one large code base, the system is split into several microservices that work together. As Maayan (2024) notes, this sort of architecture may seem a little fragmented at first but it does offer a practical way to keep services loosely coupled while still allowing each one to evolve at its own pace.

Following the database per service pattern described by Baeldung (2024) and GeeksforGeeks (2025), every microservice maintains its own datastore. This move is not without its trade-offs, maybe there's more coordinate overhead. Moreover, it generally gives each service clearer guildline and reduce the risk of cross service dependencies.

All trail metadata such as trail description or coordinates lives within this microservice ecosystem. The trend appears to reflect a broader industry move away from monolithic designs.

Relevant Software (2025) even points out that the cloud microservice market grew from £1.16 billion in 2023 and expected reach around £4.56 billion by 2030 which may suggest that organisations increasingly scalability and flexibility.

Moreover, the service use RESTful API like standard HTTP methods (GET/POST/PUT/DELETE) and JSON responses where match with the design guidelines outlined by Microsoft Learn (EdPrice-MSFT,2023).

3.0 Set Exercise 1: Normalisation

This section shows the normalisation of the Plymouth Circular trail page from AllTrails.(https://www.alltrails.com/trail/england/devon/plymbridge-circular) through to Third Normal Form(3NF). Complete tables for each stage are provided in Figure 1 below.

3.1 Assumptions Made

1) A location may exist in the database without having any trails registered to it yet
2) An owner (user) may exist in the system without having created any trails
3) A route type must be predefined in the system before trails can be assigned to it
4) Every trail must have at least one waypoint to define its path
5) Trail features are optional and can be shared across multiple trails through the junction table
6) All foreign keys enforce referential integrity with appropriate cascade rules

3.2 Extracted Attributes from Plymbridge Circular Trail

From the trail page:

- Trail name: "Plymbridge Circular"
- Rating: 4.7
- Difficulty: "Easy"
- Location: "Plymouth, Devon, England"
- Length: 5 km
- Elevation gain: 147 m
- Estimated time: 1.5-2 hour
- Loop status: Yes
- Description text
- Top sight: River Plym (River)

- Plan your visit features: Dog-friendly, Kid-friendly, Partially paved, Caves, Forests, Birding
- Weather forecast data (dates, temperatures, conditions, times)
- Nearby trails: Shaugh Bridge/Dewerstone/Cadover Circular, Saltram House Woodland and Riverside Circular, Plymbridge Old Canal and River Walk
- FAQ questions and answers

UNF (Unnormalized Form)	1NF (First Normal Form)	2NF (Second Normal Form)	3NF (Third Normal Form)	Relation Names
Trail_ID (PK)	Trail_ID (PK)	Trail_ID (PK)	Trail_ID (PK)	Trail
Trail_name	Trail_name	Trail_name	Trail_name	
Difficulty	Difficulty	Difficulty	Difficulty	
Location	Location	Location_ID (FK)	Location_ID (FK)	
Length	Length	Length	Length	
Elevation_gain	Elevation_gain	Elevation_gain	Elevation_gain	
Route_type	Route_type	Route_type	Route_ID (FK)	
Estimated_time_min	Estimated_time_min	Estimated_time_min	Estimated_time_min	
Estimated_time_max	Estimated_time_max	Estimated_time_max	Estimated_time_max	
Description_text	Description_text	Description_text	Description_text	
Overall_rating	Overall_rating	Overall_rating	Overall_rating	
Review_count	Review_count	Review_count	Review_count	
Owner_email	Owner_email	Owner_email (FK)	Owner_email (FK)	
Owner_name	Owner_name			
City	City	Location_ID (PK)	Location_ID (PK)	Location
County	County	City	City	
Country	Country	County	County	
Waypoint_sequence		Country	Country	
Waypoint_latitude	Trail_ID			
Waypoint_longitude	Waypoint_sequence	Owner_email (PK)	Owner_email (PK)	User
Waypoint_elevation	Waypoint_latitude	Owner_name	Owner_name	
Feature_name	Waypoint_longitude			
Feature_description	Waypoint_elevation	Trail_ID	Route_ID	Route_Type
		Waypoint_sequence	Route_type	
	Trail_ID	Waypoint_latitude		
	Feature_name	Waypoint_longitude	Trail_ID	Trail_Waypoint
	Feature_description	Waypoint_elevation	Waypoint_sequence	
			Waypoint_latitude	
		Feature_ID (PK)	Waypoint_longitude	
		Feature_name	Waypoint_elevation	
		Feature_description		
			Feature_ID (PK)	Feature
		Trail_ID	Feature_name	
		Feature_ID	Feature_description	
			Trail_ID	Trail_Feature
			Feature_ID	

Notation:

= Primary Key
= Foreign Key
= Repeating Group
= Composite Key

Figure 1 : All Normal Form (UNF ~3NF)

3.3 Unnormalised Form (UNF)

Initial UNF shows signs of redundancy and two repeating groups like the waypoint data (Waypoint_sequence, Waypoint_latitude, Waypoint_longitude, Waypoint_elevation) and the feature fields (Feature_name, Feature_description).

Both sets break the idea of atomicity and make the table awkward to update or store because since a trail could have many waypoint or feature.

However, the owner information like email and name or location fields like county and country inside the entity also creates update anomalies. For example, changing owner's detail would require updating all trail record that have information.

Furthermore,Overall_rating and Review_count are kept in the Trail entity because they act as summary metadata for the trail itself rather than representing individual records.

3.4 First Normal Form (1NF)

To bring into 1NF, the repeating groups were removed and the UNF was split into three separate relations so that every attribute hold only atomic values. Hence, The waypoint group was moved with a composite primary key of Trail_ID and Waypoint_sequence. Thus, every feature or waypoint avoids duplicating trail information for this seperation and properly support the relationship these attributes represent.

3.5 Second Normal Form (2NF)

Move to 2NF focused on removing partial dependencies where non-key attribute relied on only part of composite primary key instead of the whole thing.

However, feature relation shows partial dependency. Feature_description depended only on Feature_name rather than the full composite key such as Trail_ID, Feature_name. To fix this, the feature details were moved into a separate FEATURE entity with a surrogate key (Feature_ID) and a bridge table TRAIL_FEATURE was created to preserve the relationship between trails and their associated features.

Besides, similar issue appeared with the location and owner details. Repeating City, County and Country across multiple trails show avoidable redundancy, so these attributes were moved into a LOCATION entity with its own Location_ID. Likewise, owner information was refactored into a USER entity using Owner_email as the natural primary key.

3.6 Third Normal Form (3NF)

Final step to 3NF got removing transitive dependences where a non-key attribute depends on another non-key attribute rather than directly on the primary key. For example, the Trail relation occurred with Route_type. The attribute determined characteristic such as trail loops back to its starting point which means any change to route type definition would require updates across multiple trail records.

To prevent this, Route_type was moved to its own Route_type entity with Route_ID as the primary key. The trail now references definition through Route_ID foreign key. So, each relation satisfies 3NF with this adjustment now. Referential intergrity is preserved through defined foreign key relationships and redundancy is kept to minimum.

Moreover, the entity representing individuals who create trails is labelled OWNER throughout normalisation analysis where “owners” are the users who publish trail information. Thus, in partial ERD(Entity Relationship Diagram), final ERD and SQL implementation. This entity is renamed USER. The change isn’t cosmetic but it shows the multiple role of the entity within the system.

Beyond trail ownership, the USER must support general user management and role based access control e.g. admin or normal user. Therefore, this shift from OWNER to USER expands the conceptual scope while maintaining the relationship defined during normalisation.

3.7 Partial Entity Relationship Diagram

The partial Entity Relationship Diagram (ERD) below shows the entities and relationships derived from the normalizing the trail page. This partial ERD will be expanded in Exercise 2 to complete micro-service ERD.

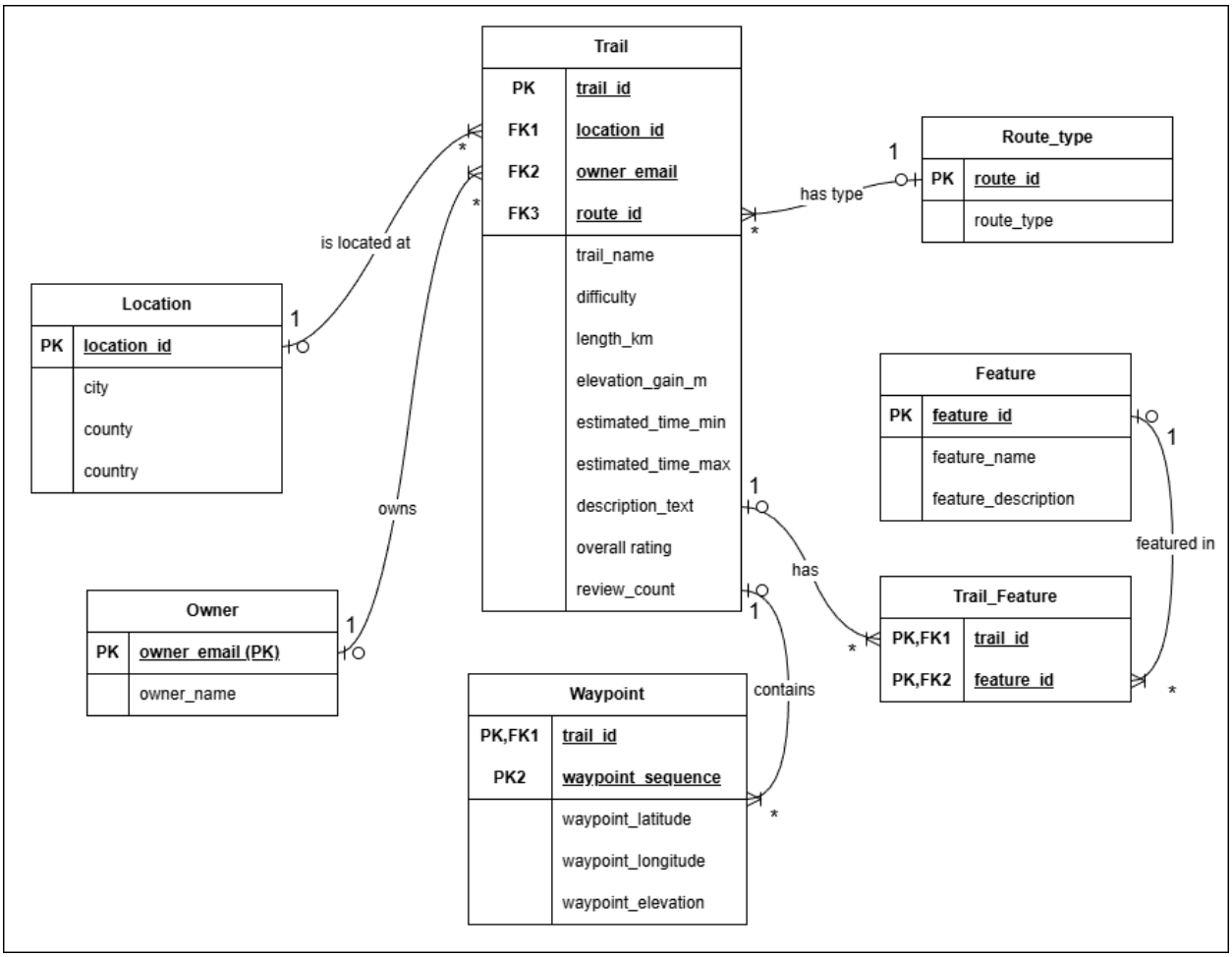


Figure 2 : Partial Entity Relationship Diagram

Notation:

PK	= Primary Key
FK	= Foreign Key
1	= One (cardinality)
*	= Many (cardinality)

4.0 Set Exercise 2: Final Entity Relationship Diagram

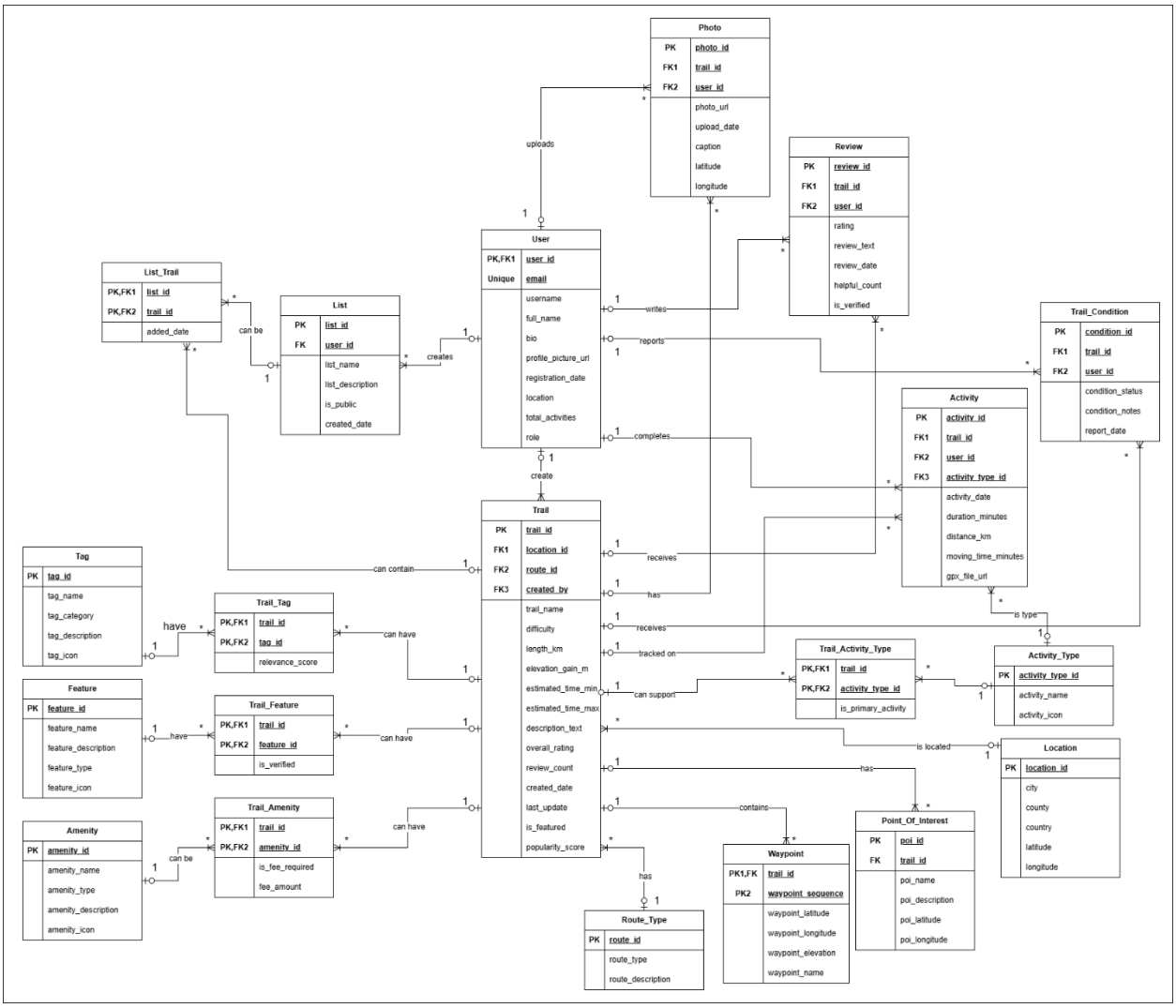


Figure 3 : Final Entity Relationship Diagram

PK = Primary Key

FK = Foreign Key

1 = One (cardinality)

* = Many (cardinality)

Cardinality Notation:

- 1:M relationships shown with "1" at parent, "M" at child
- Junction tables resolve M:M relationships with composite PKs

4.1 Final Assumptions Made

1) Users must authenticate using the external Authenticator API and cannot be deleted if they have created trails
2) Every trail must be created by exactly one user and must belong to exactly one location and route type
3) Waypoints must be ordered consecutively starting from sequence 1 for each trail
4) Review ratings are constrained to 1-5 stars and the trail's overall rating is calculated as the average of all reviews
5) User-uploaded photos and points of interest include precise latitude/longitude coordinates for geotagging
6) Trail condition reports represent real-time data submitted by users to inform others about current trail status
7) User-created lists can be either public or private and can contain any number of trails
8) Activities are independent of reviews, allowing users to record completed hikes without writing reviews
9) Junction tables use composite primary keys formed from both foreign keys to resolve many-to-many relationships
10) External resources like photos and GPX files are stored outside the database with only URLs stored in the system

5.0 Set Exercise 3: Database design

The Field Definition grids in Appendix B give specifications for each entity's attributes like data types or sizes, etc that justify implementation choices. Thus, a surrogate primary key (user_id) in the USER table is used for efficiency while the email field remains unique.

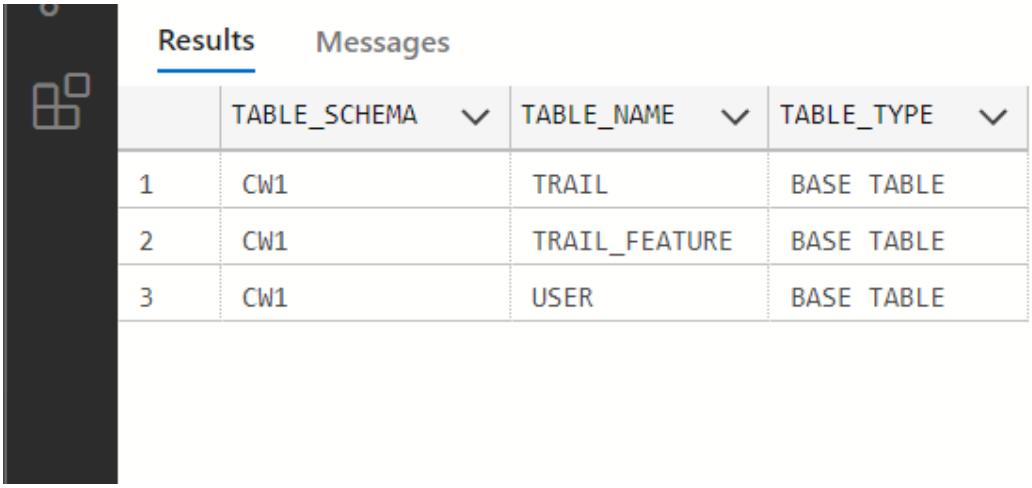
Moreover, role-based access control is implemented through a CHECK constraint that limits the role attribute to either 'admin' or 'user'.

6.0 Set Exercise 4: SQL

Screenshot 1: Schema Verification

Query:

```
SELECT TABLE_SCHEMA, TABLE_NAME, TABLE_TYPE
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'CW1'
ORDER BY TABLE_NAME;
```



Results Messages

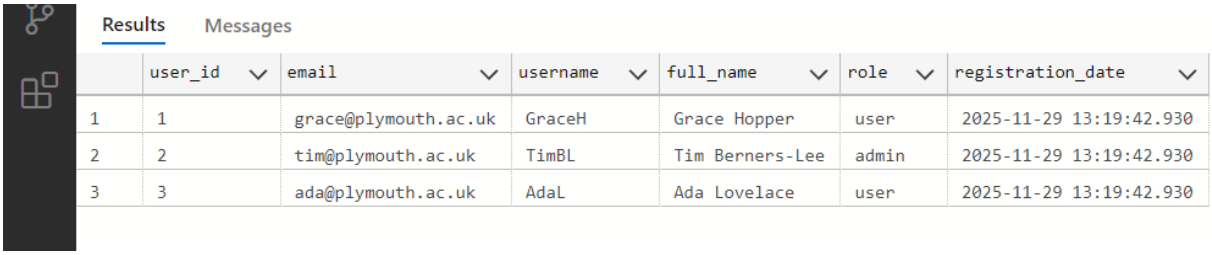
	TABLE_SCHEMA	TABLE_NAME	TABLE_TYPE
1	CW1	TRAIL	BASE TABLE
2	CW1	TRAIL_FEATURE	BASE TABLE
3	CW1	USER	BASE TABLE

Purpose: Demonstrates all three tables (USER, TRAIL, TRAIL_FEATURE) exist in CW1 schema.

Screenshot 2: USER Table Data

Query:

```
SELECT user_id, email, username, full_name, role, registration_date
FROM CW1.[USER]
ORDER BY user_id;
```



Results Messages

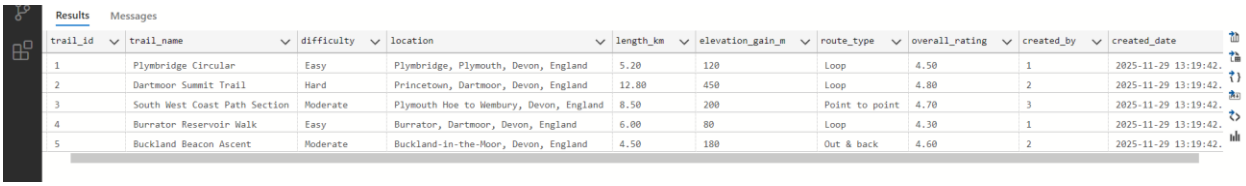
	user_id	email	username	full_name	role	registration_date
1	1	grace@plymouth.ac.uk	GraceH	Grace Hopper	user	2025-11-29 13:19:42.930
2	2	tim@plymouth.ac.uk	TimBL	Tim Berners-Lee	admin	2025-11-29 13:19:42.930
3	3	ada@plymouth.ac.uk	AdaL	Ada Lovelace	user	2025-11-29 13:19:42.930

Purpose: Shows all user records with complete data.

Screenshot 3: TRAIL Table Data

Query:

```
SELECT trail_id, trail_name, difficulty, location, length_km, elevation_gain_m,
    route_type, overall_rating, created_by, created_date
FROM CW1.TRAIL
ORDER BY trail_id;
```



Results Messages

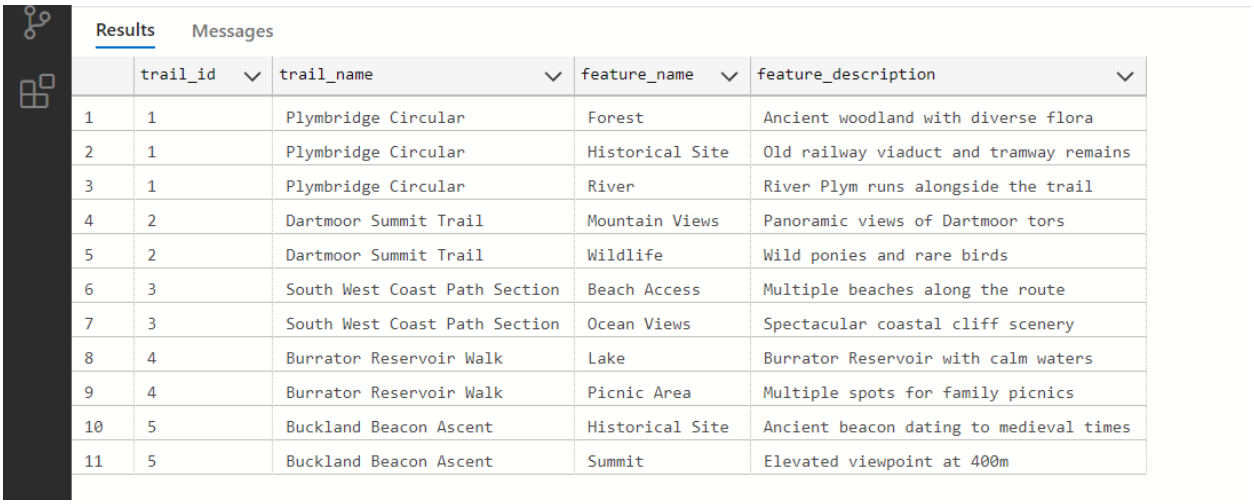
trail_id	trail_name	difficulty	location	length_km	elevation_gain_m	route_type	overall_rating	created_by	created_date
1	Plymbridge Circular	Easy	Plymbridge, Plymouth, Devon, England	5.20	120	Loop	4.50	1	2025-11-29 13:19:42.
2	Dartmoor Summit Trail	Hard	Princetown, Dartmoor, Devon, England	12.80	450	Loop	4.80	2	2025-11-29 13:19:42.
3	South West Coast Path Section	Moderate	Plymouth Hoe to Wembury, Devon, England	8.50	200	Point to point	4.70	3	2025-11-29 13:19:42.
4	Burrator Reservoir Walk	Easy	Burrator, Dartmoor, Devon, England	6.00	80	Loop	4.30	1	2025-11-29 13:19:42.
5	Buckland Beacon Ascent	Moderate	Buckland-in-the-Moor, Devon, England	4.50	180	Out & back	4.60	2	2025-11-29 13:19:42.

Purpose: Displays all five trail records demonstrating sufficient demo data for testing with varied difficulties, route types, and locations.

Screenshot 4: TRAIL_FEATURE Link Entity Data

Query:

```
SELECT tf.trail_id, t.trail_name, tf.feature_name, tf.feature_description
FROM CW1.TRAIL_FEATURE tf
INNER JOIN CW1.TRAIL t ON tf.trail_id = t.trail_id
ORDER BY tf.trail_id, tf.feature_name;
```



Results Messages

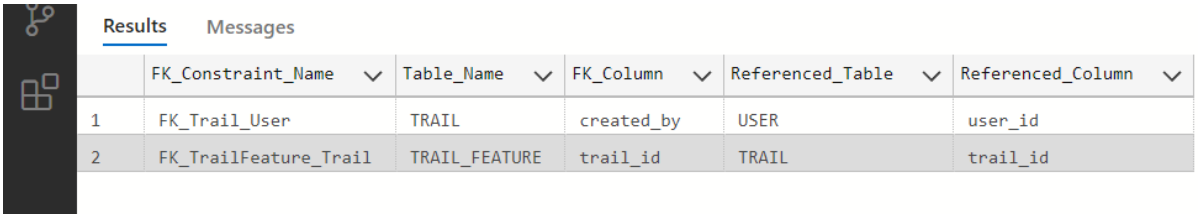
	trail_id	trail_name	feature_name	feature_description
1	1	Plymbridge Circular	Forest	Ancient woodland with diverse flora
2	1	Plymbridge Circular	Historical Site	Old railway viaduct and tramway remains
3	1	Plymbridge Circular	River	River Plym runs alongside the trail
4	2	Dartmoor Summit Trail	Mountain Views	Panoramic views of Dartmoor tors
5	2	Dartmoor Summit Trail	Wildlife	Wild ponies and rare birds
6	3	South West Coast Path Section	Beach Access	Multiple beaches along the route
7	3	South West Coast Path Section	Ocean Views	Spectacular coastal cliff scenery
8	4	Burrator Reservoir Walk	Lake	Burrator Reservoir with calm waters
9	4	Burrator Reservoir Walk	Picnic Area	Multiple spots for family picnics
10	5	Buckland Beacon Ascent	Historical Site	Ancient beacon dating to medieval times
11	5	Buckland Beacon Ascent	Summit	Elevated viewpoint at 400m

Purpose: Proves link entity implementation with eleven many-to-many relationship records showing trails with multiple features.

Screenshot 5: Foreign Key Relationships

Query:

```
SELECT fk.name AS FK_Constraint_Name,
    OBJECT_NAME(fk.parent_object_id) AS Table_Name,
    COL_NAME(fc.parent_object_id, fc.parent_column_id) AS FK_Column,
    OBJECT_NAME(fk.referenced_object_id) AS Referenced_Table,
    COL_NAME(fc.referenced_object_id, fc.referenced_column_id) AS Referenced_Column
FROM sys.foreign_keys AS fk
INNER JOIN sys.foreign_key_columns AS fc ON fk.object_id = fc.constraint_object_id
WHERE OBJECT_SCHEMA_NAME(fk.parent_object_id) = 'CW1'
ORDER BY Table_Name;
```



Results Messages

	FK_Constraint_Name	Table_Name	FK_Column	Referenced_Table	Referenced_Column
1	FK_Trail_User	TRAIL	created_by	USER	user_id
2	FK_TrailFeature_Trail	TRAIL_FEATURE	trail_id	TRAIL	trail_id

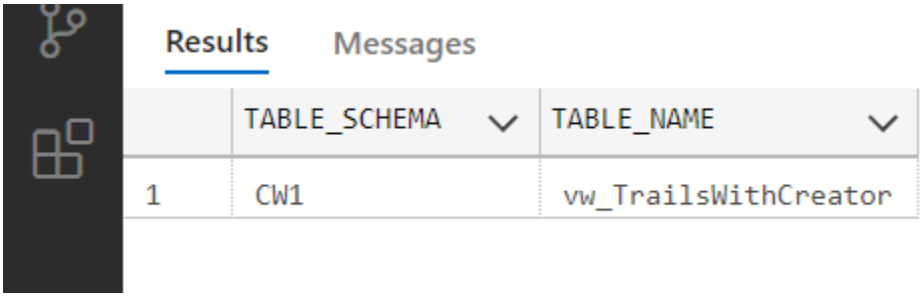
Purpose: Verifies referential integrity constraints (FK_Trail_User and FK_TrailFeature_Trail) are properly implemented.

7.0 Set Exercise 5: View

Screenshot 6: View Existence Verification

Query:

```
SELECT TABLE_SCHEMA, TABLE_NAME
FROM INFORMATION_SCHEMA.VIEWS
WHERE TABLE_SCHEMA = 'CW1' AND TABLE_NAME =
'vw_TrailsWithCreator';
```



Results Messages

	TABLE_SCHEMA	TABLE_NAME
1	CW1	vw_TrailsWithCreator

Purpose: Confirms vw_TrailsWithCreator view exists in CW1 schema.

Screenshot 7: Complete View Output

Query:

```
SELECT trail_id, trail_name, difficulty, location,
length_km, route_type,
       overall_rating, creator_username,
creator_email, creator_role, days_since_created
FROM CW1.vw_TrailsWithCreator
ORDER BY trail_id;
```

Results Messages

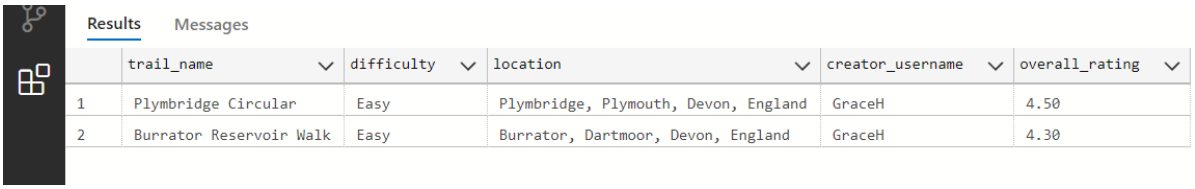
	trail_id	trail_name	difficulty	location	length_km	route_type	overall_rating	creator_username	creator_email	creator_r
1	1	Plymbridge Circular	Easy	Plymbridge, Plymouth, Devon, England	5.20	Loop	4.50	GraceH	grace@plymouth.ac.uk	user
2	2	Dartmoor Summit Trail	Hard	Princetown, Dartmoor, Devon, England	12.80	Loop	4.80	TimBL	tim@plymouth.ac.uk	admin
3	3	South West Coast Path Section	Moderate	Plymouth Hoe to Wembury, Devon, England	8.50	Point to point	4.70	AdaL	ada@plymouth.ac.uk	user
4	4	Burrator Reservoir Walk	Easy	Burrator, Dartmoor, Devon, England	6.00	Loop	4.30	GraceH	grace@plymouth.ac.uk	user
5	5	Buckland Beacon Ascent	Moderate	Buckland-in-the-Moor, Devon, England	4.50	Out & back	4.60	TimBL	tim@plymouth.ac.uk	admin

Purpose: Shows view successfully combines TRAIL and USER data with calculated field (days_since_created).

Screenshot 8: View Queryability Demonstration

Query:

```
SELECT trail_name, difficulty, location,
creator_username, overall_rating
FROM CW1.vw_TrailsWithCreator
WHERE difficulty = 'Easy'
ORDER BY overall_rating DESC;
```



Results Messages

	trail_name	difficulty	location	creator_username	overall_rating
1	Plymbridge Circular	Easy	Plymbridge, Plymouth, Devon, England	GraceH	4.50
2	Burrator Reservoir Walk	Easy	Burrator, Dartmoor, Devon, England	GraceH	4.30

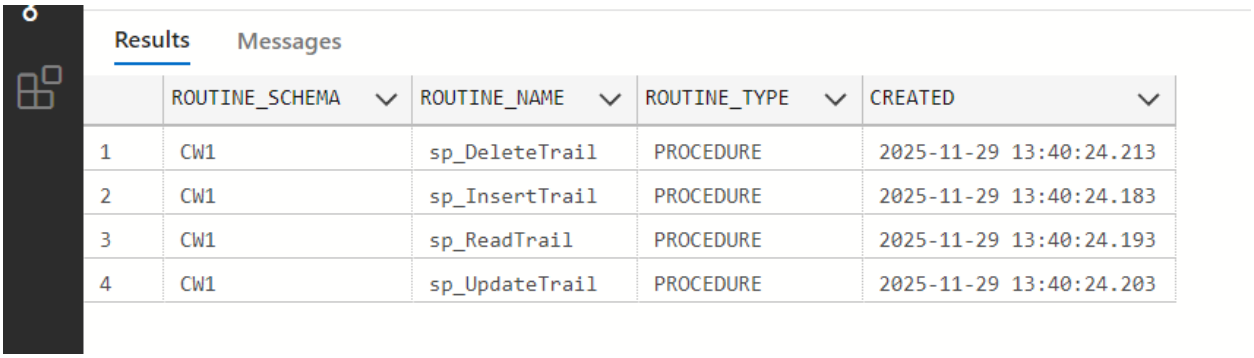
Purpose: Proves view is queryable with WHERE clause filtering by difficulty level.

8.0 Set Exercise 6: Stored procedures

Screenshot 9: Stored Procedures List

Query:

```
SELECT ROUTINE_SCHEMA, ROUTINE_NAME, ROUTINE_TYPE,
CREATED
FROM INFORMATION_SCHEMA.ROUTINES
WHERE ROUTINE_SCHEMA = 'CW1' AND ROUTINE_TYPE =
'PROCEDURE'
ORDER BY ROUTINE_NAME;
```



Results Messages

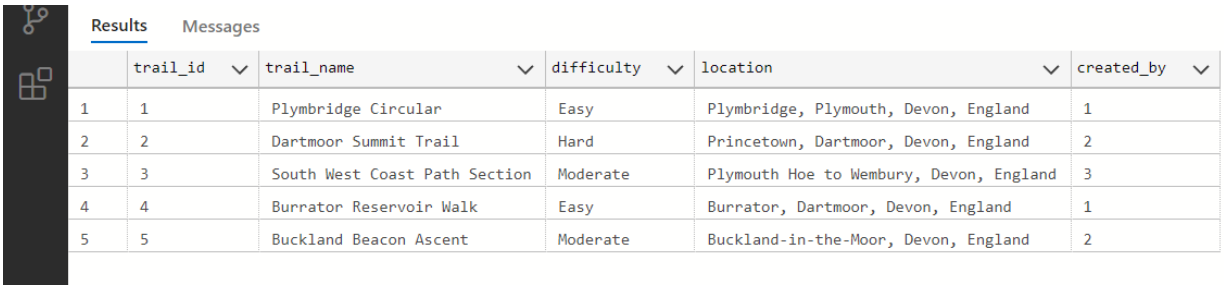
	ROUTINE_SCHEMA	ROUTINE_NAME	ROUTINE_TYPE	CREATED
1	CW1	sp_DeleteTrail	PROCEDURE	2025-11-29 13:40:24.213
2	CW1	sp_InsertTrail	PROCEDURE	2025-11-29 13:40:24.183
3	CW1	sp_ReadTrail	PROCEDURE	2025-11-29 13:40:24.193
4	CW1	sp_UpdateTrail	PROCEDURE	2025-11-29 13:40:24.203

Purpose: Confirms all four CRUD stored procedures (sp_DeleteTrail, sp_InsertTrail, sp_ReadTrail, sp_UpdateTrail) exist in CW1 schema.

Screenshot 10a: BEFORE INSERT - Current Trail Count

Query:

```
SELECT trail_id, trail_name, difficulty, location,
created_by
FROM CW1.TRAIL
ORDER BY trail_id;
```



Results Messages

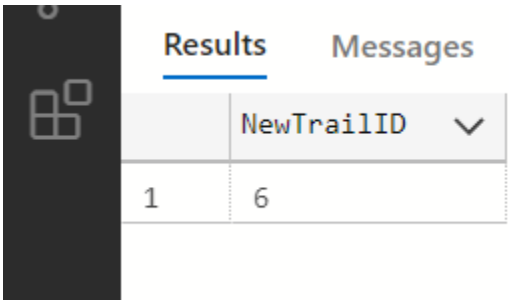
	trail_id	trail_name	difficulty	location	created_by
1	1	Plymbridge Circular	Easy	Plymbridge, Plymouth, Devon, England	1
2	2	Dartmoor Summit Trail	Hard	Princetown, Dartmoor, Devon, England	2
3	3	South West Coast Path Section	Moderate	Plymouth Hoe to Wembury, Devon, England	3
4	4	Burrator Reservoir Walk	Easy	Burrator, Dartmoor, Devon, England	1
5	5	Buckland Beacon Ascent	Moderate	Buckland-in-the-Moor, Devon, England	2

Purpose: Establishes baseline showing five existing trails before INSERT procedure execution.

Screenshot 10b: EXECUTE INSERT Procedure

Query:

```
DECLARE @new_id INT;
EXEC CW1.sp_InsertTrail
    @trail_name = 'Test Trail - Procedure Insert',
    @difficulty = 'Moderate',
    @location = 'Test Location, Devon, England',
    @length_km = 7.5,
    @elevation_gain_m = 250,
    @route_type = 'Loop',
    @description_text = 'Trail inserted via stored
procedure for testing.',
    @overall_rating = 4.20,
    @created_by = 2,
    @new_trail_id = @new_id OUTPUT;
SELECT @new_id AS NewTrailID;
```

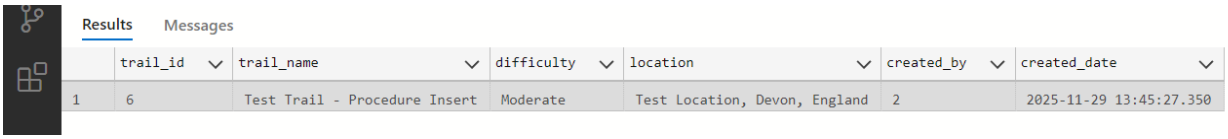


Purpose: Demonstrates INSERT procedure execution returning new trail_id (6) via OUTPUT parameter.

Screenshot 10c: AFTER INSERT - Verify New Trail

Query:

```
SELECT trail_id, trail_name, difficulty, location,
created_by, created_date
FROM CW1.TRAIL
WHERE trail_name = 'Test Trail - Procedure Insert';
```



Results Messages

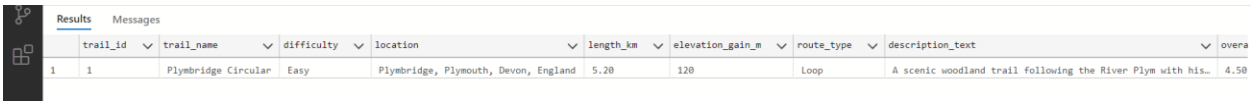
	trail_id	trail_name	difficulty	location	created_by	created_date
1	6	Test Trail - Procedure Insert	Moderate	Test Location, Devon, England	2	2025-11-29 13:45:27.350

Purpose: Confirms new trail (ID 6) was inserted successfully with correct attributes.

Screenshot 11a: READ Specific Trail

Query:

```
EXEC CW1.sp_ReadTrail @trail_id = 1;
```



Results Messages

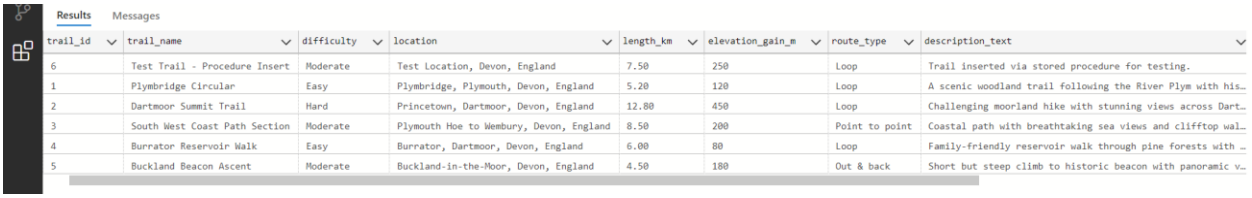
	trail_id	trail_name	difficulty	location	length_km	elevation_gain_m	route_type	description_text	overa
1	1	Plymbridge Circular	Easy	Plymbridge, Plymouth, Devon, England	5.20	120	Loop	A scenic woodland trail following the River Plym with his…	4.50

Purpose: Shows READ procedure retrieves specific trail (Plymbridge Circular) by ID with all columns.

Screenshot 11b: READ All Trails

Query:

```
EXEC CW1.sp_ReadTrail;
```



Results Messages

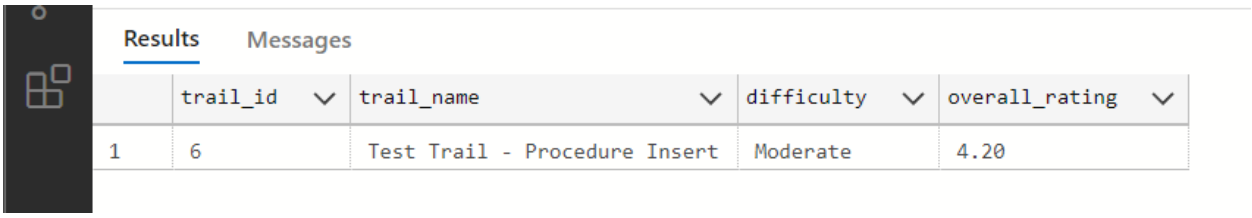
trail_id	trail_name	difficulty	location	length_km	elevation_gain_m	route_type	description_text
6	Test Trail - Procedure Insert	Moderate	Test Location, Devon, England	7.50	250	Loop	Trail inserted via stored procedure for testing.
1	Plymbridge Circular	Easy	Plymbridge, Plymouth, Devon, England	5.20	120	Loop	A scenic woodland trail following the River Plym with his…
2	Dartmoor Summit Trail	Hard	Princetown, Dartmoor, Devon, England	12.80	450	Loop	Challenging moorland hike with stunning views across Dart…
3	South West Coast Path Section	Moderate	Plymouth Hoe to Wembury, Devon, England	8.50	200	Point to point	Coastal path with breathtaking sea views and clifftop wal…
4	Burrator Reservoir Walk	Easy	Burrator, Dartmoor, Devon, England	6.00	80	Loop	Family-friendly reservoir walk through pine forests with …
5	Buckland Beacon Ascent	Moderate	Buckland-in-the-Moor, Devon, England	4.50	180	Out & back	Short but steep climb to historic beacon with panoramic v…

Purpose: Demonstrates READ procedure returns all six trails when no parameter provided, ordered by created_date DESC.

Screenshot 12a: BEFORE UPDATE - Original Data

Query:

```
SELECT trail_id, trail_name, difficulty,
overall_rating
FROM CW1.TRAIL
WHERE trail_id = 6;
```



Results Messages

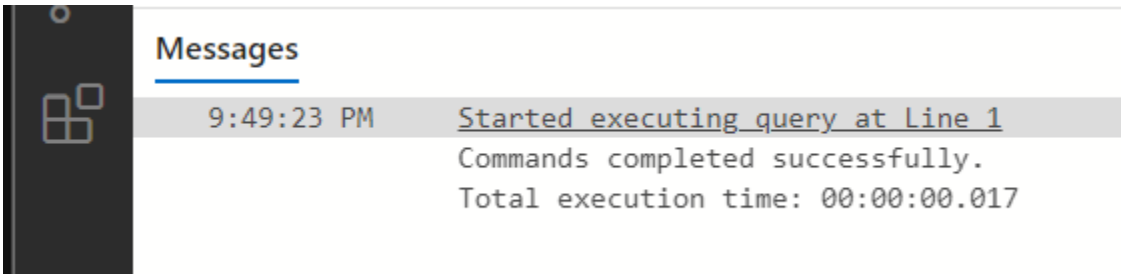
	trail_id	trail_name	difficulty	overall_rating
1	6	Test Trail - Procedure Insert	Moderate	4.20

Purpose: Shows original trail data (Test Trail - Procedure Insert, Moderate, 4.20) before UPDATE operation.

Screenshot 12b: EXECUTE UPDATE Procedure

Query:

```
EXEC CW1.sp_UpdateTrail
    @trail_id = 6,
    @trail_name = 'Test Trail - UPDATED',
    @difficulty = 'Hard',
    @overall_rating = 4.75;
```

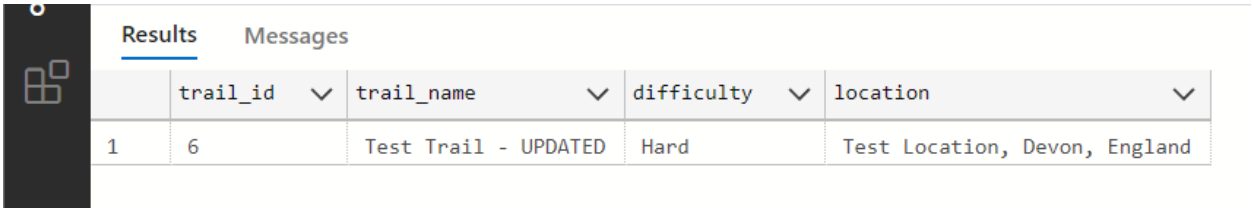


Purpose: Demonstrates UPDATE procedure execution with partial field updates using ISNULL logic.

Screenshot 12c: AFTER UPDATE - Modified Data

Query:

```
SELECT trail_id, trail_name, difficulty,
overall_rating
FROM CW1.TRAIL
WHERE trail_id = 6;
```

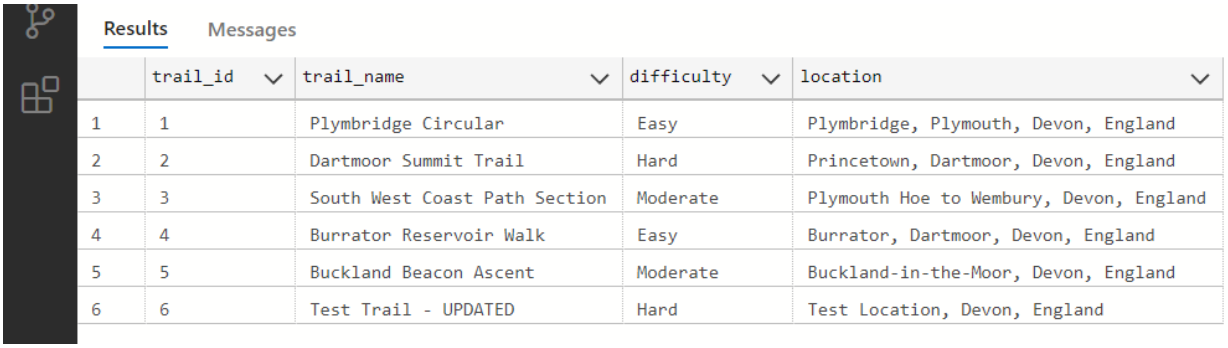


Purpose: Confirms trail data was updated correctly (Test Trail - UPDATED, Hard, 4.75).

Screenshot 13a: BEFORE DELETE - Trail Exists

Query:

```
SELECT trail_id, trail_name, difficulty, location
FROM CW1.TRAIL
```



Results | Messages

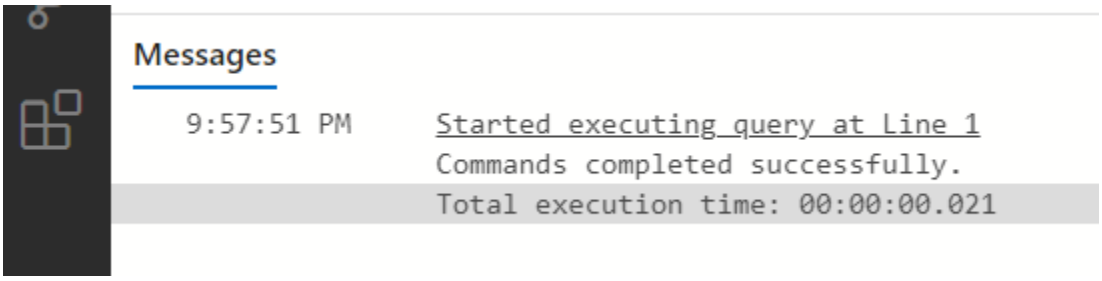
	trail_id	trail_name	difficulty	location
1	1	Plymbridge Circular	Easy	Plymbridge, Plymouth, Devon, England
2	2	Dartmoor Summit Trail	Hard	Princetown, Dartmoor, Devon, England
3	3	South West Coast Path Section	Moderate	Plymouth Hoe to Wembury, Devon, England
4	4	Burrator Reservoir Walk	Easy	Burrator, Dartmoor, Devon, England
5	5	Buckland Beacon Ascent	Moderate	Buckland-in-the-Moor, Devon, England
6	6	Test Trail - UPDATED	Hard	Test Location, Devon, England

Purpose: Shows trail ID 6 exists with complete data before DELETE operation.

Screenshot 13b: EXECUTE DELETE Procedure

Query:

```
EXEC CW1.sp_DeleteTrail @trail_id = 6;
```

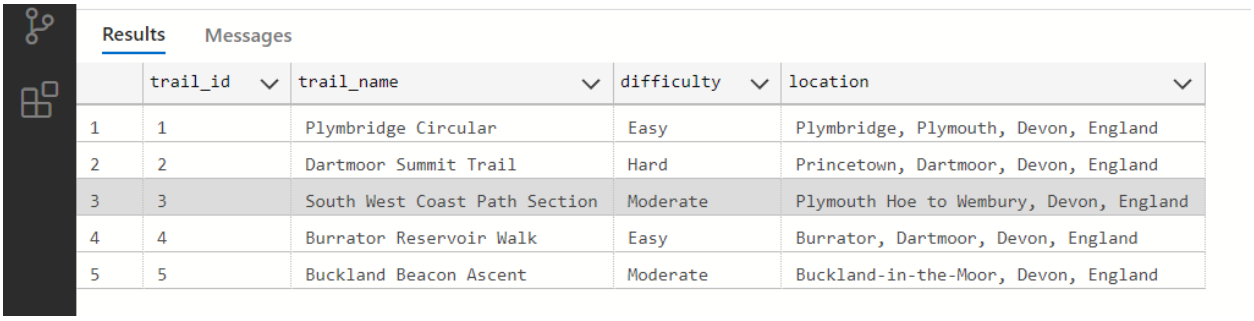


Purpose: Demonstrates DELETE procedure execution with CASCADE effect on TRAIL_FEATURE records.

Screenshot 13c: AFTER DELETE - Trail Removed

Query:

```
SELECT trail_id, trail_name, difficulty, location
FROM CW1.TRAIL
```



Results Messages

	trail_id	trail_name	difficulty	location
1	1	Plymbridge Circular	Easy	Plymbridge, Plymouth, Devon, England
2	2	Dartmoor Summit Trail	Hard	Princetown, Dartmoor, Devon, England
3	3	South West Coast Path Section	Moderate	Plymouth Hoe to Wembury, Devon, England
4	4	Burrator Reservoir Walk	Easy	Burrator, Dartmoor, Devon, England
5	5	Buckland Beacon Ascent	Moderate	Buckland-in-the-Moor, Devon, England

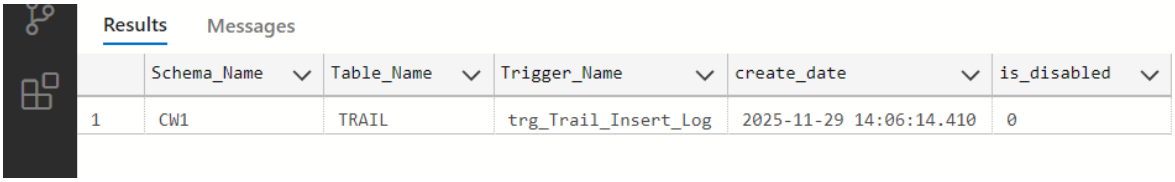
Purpose: Confirms trail ID 6 was deleted successfully.

9.0 Set Exercise 7: Trigger

Screenshot 14: Trigger Existence Verification

Query:

```
SELECT s.name AS Schema_Name, t.name AS Table_Name,
tr.name AS Trigger_Name,
       tr.create_date, tr.is_disabled
FROM sys.triggers tr
INNER JOIN sys.tables t ON tr.parent_id =
t.object_id
INNER JOIN sys.schemas s ON t.schema_id =
s.schema_id
WHERE s.name = 'CW1' AND tr.name =
'trg_Trail_Insert_Log';
```



Results Messages

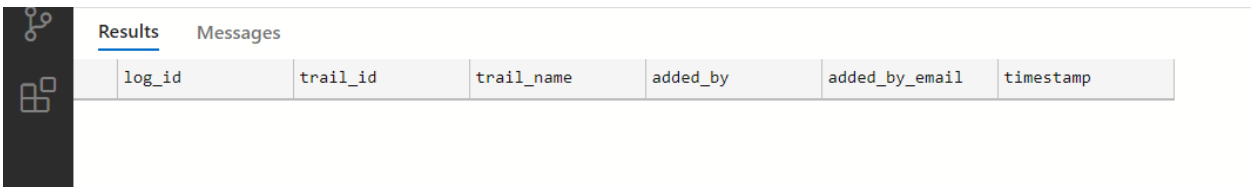
	Schema_Name	Table_Name	Trigger_Name	create_date	is_disabled
1	CW1	TRAIL	trg_Trail_Insert_Log	2025-11-29 14:06:14.410	0

Purpose: Confirms trg_Trail_Insert_Log trigger exists on TRAIL table in CW1 schema with is_disabled status false.

Screenshot 15a: BEFORE INSERT - Initial Log State

Query:

```
SELECT log_id, trail_id, trail_name, added_by,
added_by_email, timestamp
FROM CW1.TRAIL_LOG
ORDER BY log_id DESC;
```



Results Messages

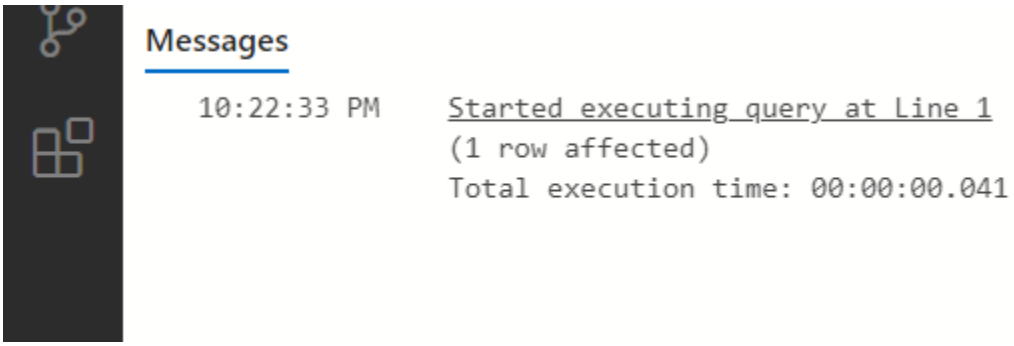
log_id	trail_id	trail_name	added_by	added_by_email	timestamp

Purpose: Shows TRAIL_LOG table state before trigger test establishing empty baseline or previous log entries.

Screenshot 15b: INSERT Trail - Trigger Fires

Query:

```
INSERT INTO CW1.TRAIL (trail_name, difficulty,
location, length_km, elevation_gain_m,
                       route_type,
description_text, overall_rating, created_by)
VALUES ('Trigger Test Trail', 'Easy', 'Trigger Test
Location, Devon, England',
        3.5, 50, 'Loop', 'Trail inserted to test
trigger functionality.', 4.00, 1);
```

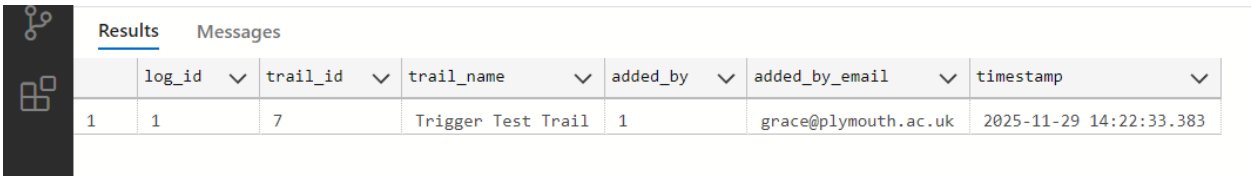


Purpose: Executes INSERT operation that automatically fires AFTER INSERT trigger for audit logging.

Screenshot 15c: AFTER INSERT - New Log Entry

Query:

```
SELECT log_id, trail_id, trail_name, added_by,
added_by_email, timestamp
FROM CW1.TRAIL_LOG
ORDER BY log_id DESC;
```



Results Messages

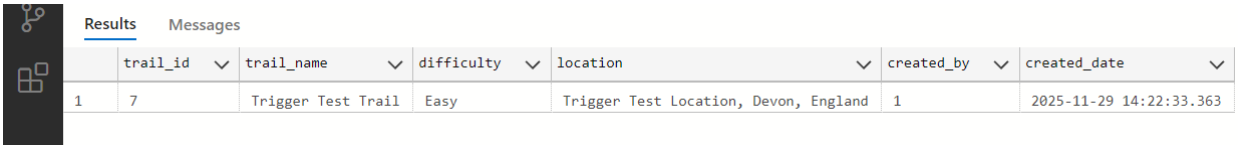
	log_id	trail_id	trail_name	added_by	added_by_email	timestamp
1	1	7	Trigger Test Trail	1	grace@plymouth.ac.uk	2025-11-29 14:22:33.383

Purpose: Verifies trigger created audit log entry automatically capturing trail_id, trail_name, user_id (1), and grace@plymouth.ac.uk.

Screenshot 15d: Verify Trail in TRAIL Table

Query:

```
SELECT trail_id, trail_name, difficulty, location,
created_by, created_date
FROM CW1.TRAIL
WHERE trail_name = 'Trigger Test Trail';
```



Results Messages

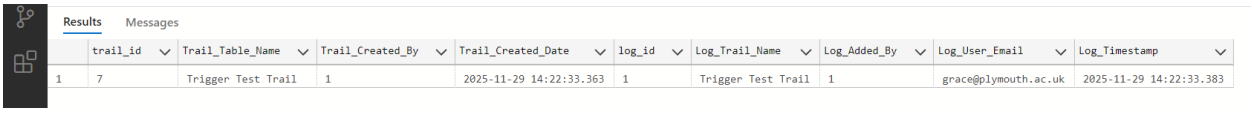
	trail_id	trail_name	difficulty	location	created_by	created_date
1	7	Trigger Test Trail	Easy	Trigger Test Location, Devon, England	1	2025-11-29 14:22:33.363

Purpose: Confirms Trigger Test Trail was inserted successfully in parent TRAIL table with all attributes.

Screenshot 15e: JOIN - Correlation Verification

Query:

```
SELECT t.trail_id, t.trail_name AS
Trail_Table_Name, t.created_by AS Trail_Created_By,
       t.created_date AS Trail_Created_Date,
tl.log_id, tl.trail_name AS Log_Trail_Name,
       tl.added_by AS Log_Added_By,
tl.added_by_email AS Log_User_Email,
       tl.timestamp AS Log_Timestamp
FROM CW1.TRAIL t
INNER JOIN CW1.TRAIL_LOG tl ON t.trail_id =
tl.trail_id
WHERE t.trail_name = 'Trigger Test Trail';
```



Results Messages

	trail_id	Trail_Table_Name	Trail_Created_By	Trail_Created_Date	log_id	Log_Trail_Name	Log_Added_By	Log_User_Email	Log_Timestamp
1	7	Trigger Test Trail	1	2025-11-29 14:22:33.363	1	Trigger Test Trail	1	grace@plymouth.ac.uk	2025-11-29 14:22:33.383

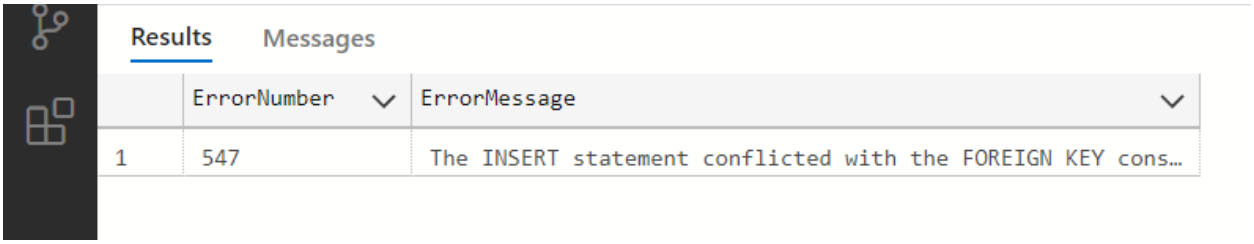
Purpose: Demonstrates audit log data matches source trail data perfectly with identical trail_id, trail_name, and user information.

ADDITIONAL VERIFICATION SCREENSHOTS

Screenshot 16: Foreign Key Constraint Error

Query:

```
BEGIN TRY
    INSERT INTO CW1.TRAIL (trail_name, difficulty,
location, length_km, route_type, created_by)
    VALUES ('Invalid Trail', 'Easy', 'Test', 5.0,
'Loop', 9999);
END TRY
BEGIN CATCH
    SELECT ERROR_NUMBER() AS ErrorNumber,
ERROR_MESSAGE() AS ErrorMessage;
END CATCH;
```

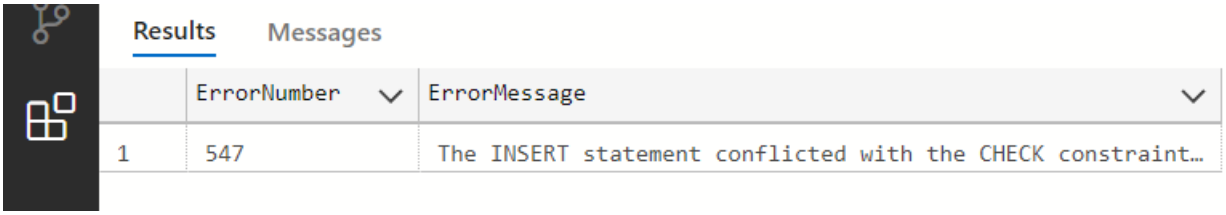


Purpose: Proves FK_Trail_User foreign key constraint prevents insertion with non-existent user_id (9999) returning error 547.

Screenshot 17: CHECK Constraint Error

Query:

```
BEGIN TRY
    INSERT INTO CW1.TRAIL (trail_name, difficulty,
location, length_km, route_type, created_by)
    VALUES ('Invalid Difficulty', 'Super Hard',
'Test', 5.0, 'Loop', 1);
END TRY
BEGIN CATCH
    SELECT ERROR_NUMBER() AS ErrorNumber,
ERROR_MESSAGE() AS ErrorMessage;
END CATCH;
```

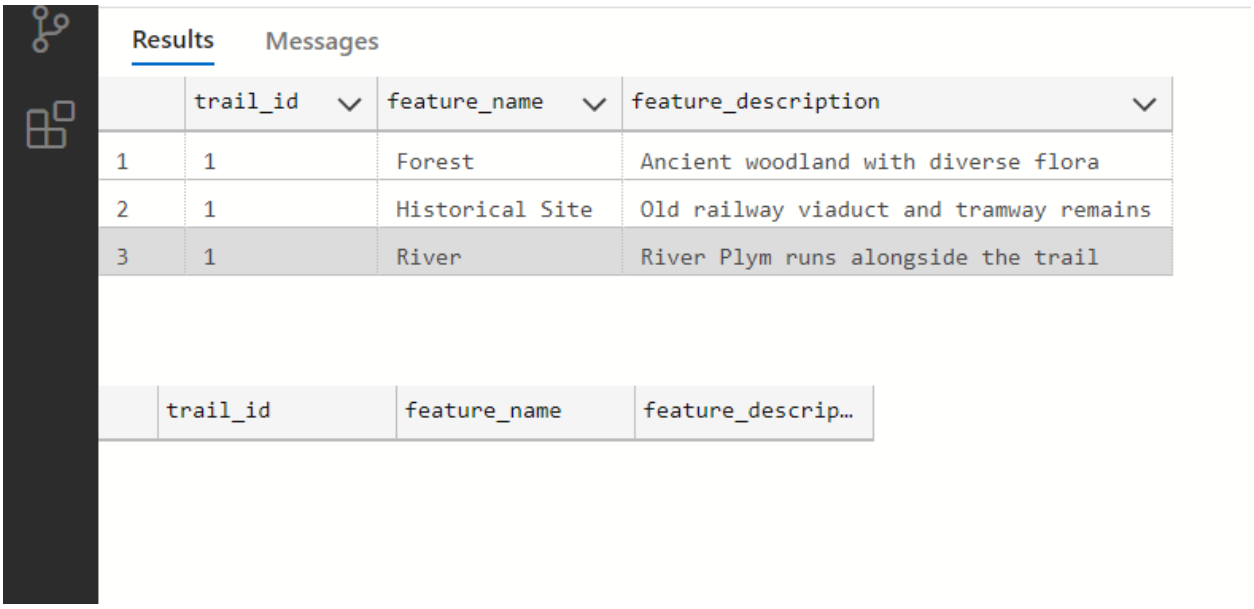


Purpose: Demonstrates CK_Trail_Difficulty CHECK constraint rejects invalid difficulty value (Super Hard) returning error 547.

Screenshot 18: CASCADE DELETE Test

Query:

```
SELECT * FROM CW1.TRAIL_FEATURE WHERE trail_id = 1;
DELETE FROM CW1.TRAIL WHERE trail_id = 1;
SELECT * FROM CW1.TRAIL_FEATURE WHERE trail_id = 1;
```



Results Messages

	trail_id	feature_name	feature_description
1	1	Forest	Ancient woodland with diverse flora
2	1	Historical Site	Old railway viaduct and tramway remains
3	1	River	River Plym runs alongside the trail

	trail_id	feature_name	feature_descrip…

Purpose: Proves ON DELETE CASCADE in FK_TrailFeature_Trail automatically removes three related TRAIL_FEATURE records when parent trail deleted.

10.0 Conclusion

This coursework demonstrated the relational database design lifecycle for the TrailService microservice.It systematically applying normalisation principles from UNF through to Third Normal Form to eliminate data redundancy and maintain referential integrity. The implementation of views, stored procedures and triggers in Microsoft SQL Server provides a foundation that encapsulates business logic and enables secure CRUD operations through parameterised queries that mitigate SQL injection vulnerabilities.

11.0 References

Draw.io - free flowchart maker and diagrams online (no date) Flowchart Maker & Online Diagram Software. Available at: https://app.diagrams.net/?libs=general%3Ber# (Accessed: 19 November 2025).

Youtu.be. (2025). Available at: https://youtu.be/busnzKKSOxU?si=mY59EIeLlfwq_PSR (Accessed 29 Nov. 2025)

Stec, A. (2024). Database Design in a Microservices Architecture | Baeldung on Computer Science. [online] www.baeldung.com. Available at: https://www.baeldung.com/cs/microservices-db-design. (Accessed: 20 November 2025).

GeeksforGeeks (2025). Database Per Service Pattern for Microservices. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/system-design/database-per-service-pattern-for-microservices/. (Accessed: 20 November 2025).

Maayan, G.D. (2024). Best of 2023: 5 Microservices Design Patterns Every DevOps Team Should Know. [online] DevOps.com. Available at: https://devops.com/5-microservices-design-patterns-every-devops-team-should-know/. (Accessed: 21 November 2025).

EdPrice-MSFT (2023). Web API design best practices - Azure Architecture Center. [online] learn.microsoft.com. Available at: https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design. (Accessed: 21 November 2025).

Relevant Software. (2025). All You Need to Know About Microservices Database Management. [online] Available at: https://relevant.software/blog/microservices-database-management/. (Accessed: 22 November 2025).

12.0 Appendices

Appendix A : Initial ERD

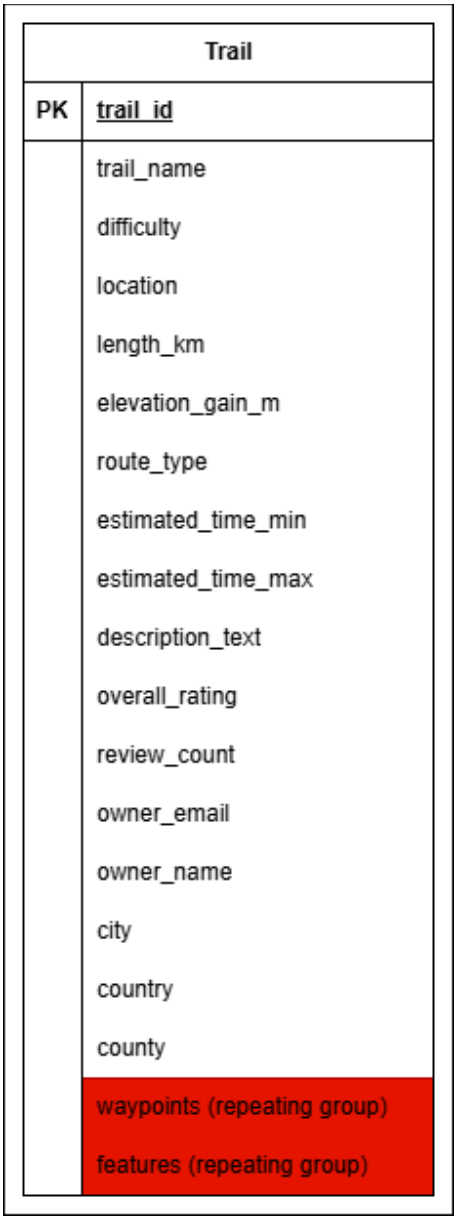


Figure 4: Initial Entity Relationship Diagram

Assumptions about Initial ERD:

Trail is the central enity containing all trail information. However, the waypoints and features are stored as repeating groups. Then, the owner information is embedded within the trail entity. Last, location data such as city, county, country is denormalized.

Appendix B : Field Definition Grids

Table 1: USER

Field Name	Data Type	Size	Constraints	Description
user_id	INT	-	PK, IDENTITY(1,1)	Unique user identifier
Email	NVARCHAR	255	NOT NULL, UNIQUE	User email (matches auth API)
Username	NVARCHAR	100	NOT NULL	Display name
full_name	NVARCHAR	255	NULL	User's full name
Role	NVARCHAR	50	NOT NULL, DEFAULT 'user'	User role (admin/user)
registration_date	DATETIME	-	NOT NULL, DEFAULT GETDATE()	Account creation timestamp

Table 2: TRAIL

Field Name	Data Type	Size	Constraints	Description
trail_id	INT	-	PK, IDENTITY(1,1)	Unique trail identifier
trail_name	NVARCHAR	255	NOT NULL	Trail name
difficulty	NVARCHAR	50	NOT NULL, CHECK	Easy/Moderate/Hard
location	NVARCHAR	255	NOT NULL	Trail location description
length_km	DECIMAL	10,2	NOT NULL, CHECK > 0	Trail length in kilometers
elevation_gain_m	INT	-	NULL, CHECK >= 0	Elevation gain in meters
route_type	NVARCHAR	50	NOT NULL	Loop/Out & back/Point to point
description_text	NVARCHAR	MAX	NULL	Trail description

Field Name	Data Type	Size	Constraints	Description
overall_rating	DECIMAL	3,2	NULL, CHECK 1-5	Average rating
created_by	INT	-	NOT NULL, FK → USER	Trail creator user_id
created_date	DATETIME	-	NOT NULL, DEFAULT GETDATE()	Creation timestamp

Table 3: TRAIL_FEATURE

Field Name	Data Type	Size	Constraints	Description
trail_id	INT	-	PK, FK → TRAIL	Reference to trail
feature_name	NVARCHAR	100	PK	Feature name
feature_description	NVARCHAR	500	NULL	Feature description

Table 4: TRAIL_LOG (for Trigger)

Field Name	Data Type	Size	Constraints	Description
log_id	INT	-	PK, IDENTITY(1,1)	Unique log identifier
trail_id	INT	-	NOT NULL	Trail that was added
trail_name	NVARCHAR	255	NOT NULL	Name of trail added
added_by	INT	-	NOT NULL	User who added trail
added_by_email	NVARCHAR	255	NOT NULL	Email of user
timestamp	DATETIME	-	NOT NULL, DEFAULT GETDATE()	When trail was added